Clase 6: Clases y polimorfismo

Nicolás Alejandro Barnafi Wittwer
Pontificia Universidad Católica de Chile
Centro de Modelamiento Matemático

Contents

1. Random, Statistics
2. Scipy
3. Pandas

Clases y objetos

Motivación

```
>>> df = pd.DataFrame() # Constructor
>>> type(df)
  <class 'pandas.core.frame.DataFrame'>
```

- Crear nuestros propios tipos de datos
- Crear abstracciones

$$\text{animal} \in \text{ser vivo}, \text{cuadrado} \in \text{figura}$$

- *Programación Orientada a Objetos* (OOP)

Clases conocidas

```
>>> type(pd.DataFrame()) # pandas
    <class 'pandas.core.frame.DataFrame'>
>>> type(np.linspace(0,1)) # numpy
    <class 'numpy.ndarray'>
>>> type(2) # base
    <class 'int'>
```

Sintaxis

```
class A:
    # Método constructor
    # self es obligatorio
    def __init__(self, n):
        self.n = n # son distintos!
        self.n2 = n**2
    def saludar(self): print("hola")

a = A(4) # Constructor
print(a.n, a.n2) # 4 16
print(type(a)) # <class '__main__.A'>
```

Ejemplo: vector 2D

```
class Vec2D:
   # Método constructor
   # self es obligatorio
   def __init__(self, x, y): # magic function
      self.x = x
      self.y = y
   def __add__(self, v2): # magic function
      out = Vec2D(self.x, self.y) # copiar
      out.x += v2.x # agregar nuevos valores
      out.y += v2.y
      return out # retornar copia
   def __getitem__(self, i): # magic function
      if i == 0: return self.x
      if i == 1: return self.y
      else:
         print("ERROR") # i = 0 o 1
```

Consola

Contents

1. Random, Statistics
2. Scipy
3. Pandas

Polimorfismo

Motivación

- Abstraer "Figura"

$$\text{animal} \in \text{ser vivo}, \text{ cuadrado} \in \text{figura}$$

- Sistema ecológico (pizarra)

$$\text{Carnívoro} \quad \rightarrow \quad \text{Herbívoro} \quad \rightarrow \quad \text{Plantas}$$

Sintaxis

```
class ClaseHeredada(ClaseBase):
    def funcion(self):
        # [...]
    # Opcional
    def __init__(self, params):
        super(ClaseHeredada, self).__init__(params)
```

Ejemplo I (src: HowToGeek)

```
class Bird:
  def intro(self): print("There are many types of birds.")

  def flight(self): print("Most of the birds can fly but some cannot.")

class sparrow(Bird):
  def flight(self): print("Sparrows can fly.")

class ostrich(Bird):
  def flight(self): print("Ostriches cannot fly.")

obj_bird = Bird(); obj_spr = sparrow(); obj_ost = ostrich()
obj_bird.flight(); obj_spr.flight(); obj_ost.flight()
```

Ejemplo II (src: HowToGeek)

```python
class Animal:
    def speak(self):
        raise NotImplementedError("Subclass must implement this method")

class Dog(Animal):
    def speak(self): return "Woof!"

class Cat(Animal):
    def speak(self): return "Meow!"

animals = [Dog(), Cat()]

for animal in animals:
    print(animal.speak())
```

Clase 6: Clases y polimorfismo

Nicolás Alejandro Barnafi Wittwer
Pontificia Universidad Católica de Chile
Centro de Modelamiento Matemático

Contents

1. Random, Statistics
2. Scipy
3. Pandas

Documentación

Motivación

- El código *envejece*
- Uno **no** recuerda por qué hizo cosas
- Los futuros usuarios de nuestro código menos

Docs de una función

```
def suma(x,y):
    """
    Esta función entrega la suma de sus argumentos.
    args:
        x: Un elemento sumable
        y: Un segundo elemento sumable
    Return: suma de entradas (Int)
    """
    z = x+y
    return z
```

Docs de una función

```
>>> help(suma)
```

Se guarda en la variable mágica `suma.__doc__`

Consola

Docs de una clase

```
class Vec2D:
    """
    Clase que representa un vector en R2.
    """
    def __init__(self,x,y):
        """
        Constructor, toma como parámetros coordenadas x e y.
        """
        self.x = x
        self.y = y
    def norma(self):
        """
        Entrega la norma euclideana del vector. Requiere 'sqrt'
        """
        return sqrt(self.x**2 + self.y**2)
```

Consola

Documentación avanzada

Existe mucho software (Sphinx, Doxygen, etc). Veremos `Mkdocs`.

```
# pip install mkdocs
# mkdocs new proyecto
# cd proyecto
# mkdocs serve
# mkdocs build
```

Se usa formato Markdown (popular por ser simple)

Mkdocs

Veamos su documentación:
https://www.mkdocs.org/getting-started/

Otra opción

```
# pip install pdoc3
# cd proyecto
# pdoc3 --html .
```

Consola

Contents

1. Random, Statistics
2. Scipy
3. Pandas

Testing

Motivación

1. Agrego cambio en función
2. Pruebo nuevo código
3. Falla un caso de borde escondidísimo

Tener una cadena de pruebas automáticas

Solución: Pytest

```
> myLib.py
def suma(a,b): return a+b

> test.py
from myLib import suma

def test_suma(): #debe comenzar con 'test'
    assert suma(1,2) == 3
$ pytest test.py
```

Recap

- Documentación es fundamental
- Existen muchas herramientas para hacerlo en modo semi-automático
- `mkdocs` bonito pero trabajoso
- `pdoc3` feo pero fácil
- `git` es LA herramienta de desarrollo colaborativo
- Se basa en un grafo de estados
- Modificaciones locales: `add`, `commit`, `reset`
- Modificaciones remotas: `clone`, `pull`, `push`

Mini ejercicios

1. Instalar git
2. Crear cuenta en `github.com`
3. Subir llave SSH
4. Crear repositorio en su cuenta
5. Clonar el repositorio localmente
6. Crear una carpeta de nombre 'proyecto'
7. Dentro esa carpeta crear un archivo con dos funciones
8. Instalar pdoc3 y correr `pdoc3 --html .` desde la carpeta
9. Ver documentación en archivo html recién creado
10. Hacer *push* del cambio y verificar el cambio

Turorial online: `https://www.w3schools.com/git/`

Recap

- Clases permiten crear abstracciones
- Todo objeto es una instancia de clase en Python
- Funciones mágicas permiten código legible!

Clase 6: Clases y polimorfismo

Nicolás Alejandro Barnafi Wittwer
Pontificia Universidad Católica de Chile
Centro de Modelamiento Matemático

Mini ejercicios

- Crear una clase Cuadrado que reciba como entrada el largo de sus aristas. Cree una función de clase que entregue el área del cuadrado.
- Extienda el ejemplo anterior al caso de un cubo.
- Instalar pdoc3 y correr `pdoc3 --html .` desde la carpeta y revisar documentación.

```
class Cuadrado:
    def __init__(self, lado): # [... implementación ...]
    def area(self):  # [... implementación ...]
```

- Explore el siguiente código:

```
class A:
    def a(self): print("Aa")

class B:
    def a(self):
        A.a(self)
        print("Ba")
```